TRANSPORT LAYER

CIS*3210
Fall 2026

Sajid Marhon

Based on notes for Computer Networking: A Top-Down Approach, 9th ed., by J. Kurose and K. Ross

Learning Outcomes



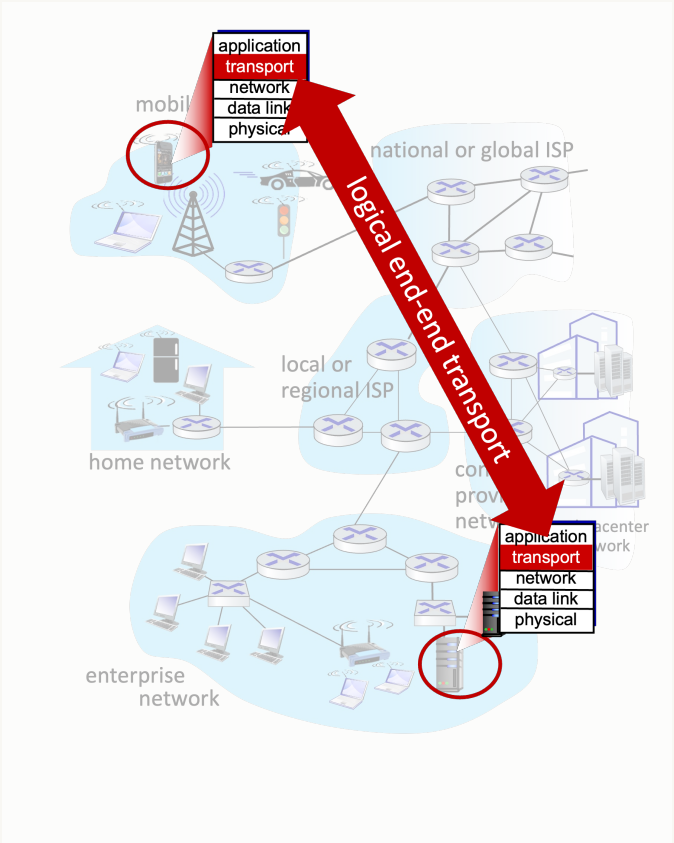
By the end of this lesson, students will be able to:

- Explain how the transport layer enables process-to-process communication using multiplexing, demultiplexing, and port numbers
- Compare UDP and TCP services, including connection establishment, reliability, and control
- Describe how the transport layer builds on the network layer to provide services to applications

Transport Layer



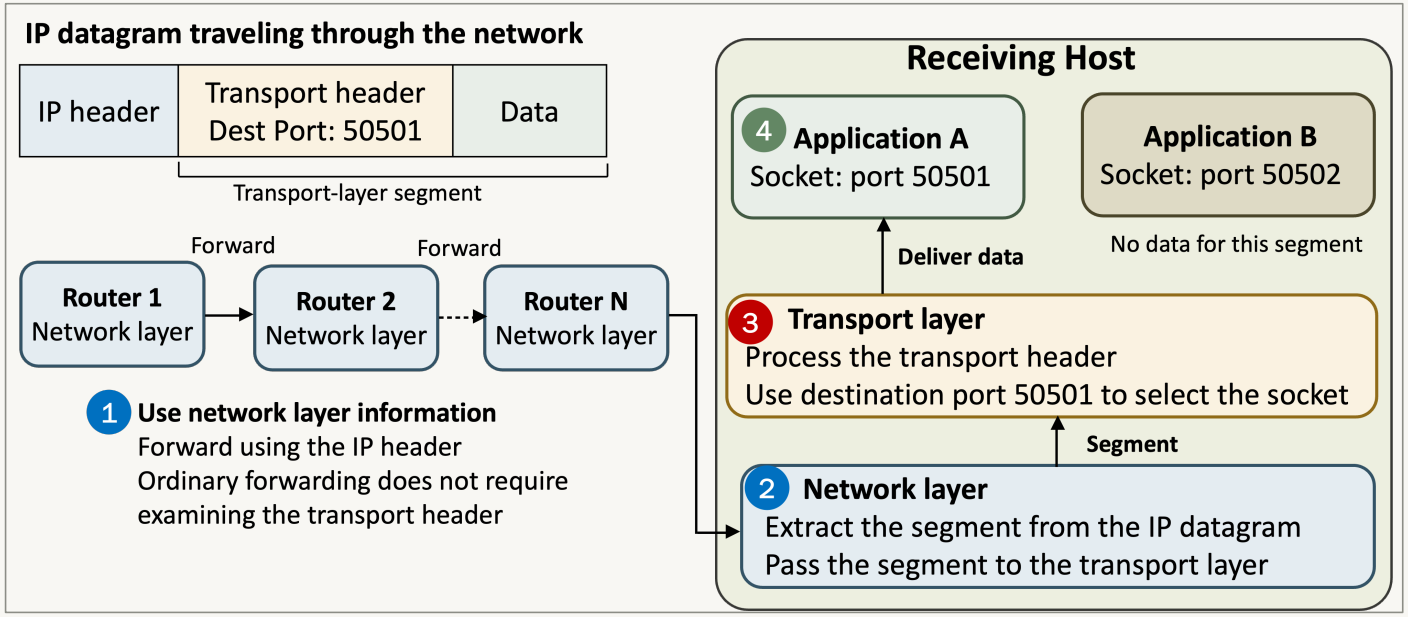
- **Logical communication:** Makes processes on different hosts appear directly connected
- **Abstraction:** Hides the underlying network infrastructure from applications
- **Implementation:** Transport protocols run on end systems, not routers
- **Sending process:**
  - Divides application messages into chunks if needed
  - Adds a transport header to form a **segment**
  - Passes the segment to the network layer for encapsulation and delivery



Transport Layer



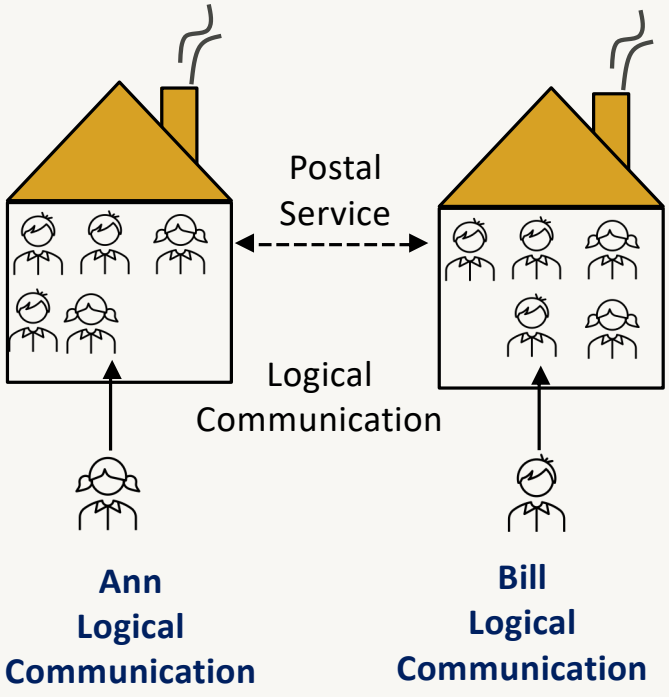
- **Router role**: Routers only process network-layer fields in the datagram; they do not inspect transport-layer headers
- **Receiving process**:
  - The network layer extracts the transport-layer segment from the datagram
  - The transport layer processes the segment and delivers data to the receiving application
- **Multiple protocols**: The Internet uses **TCP** and **UDP**, each offering different transport services to applications



Transport vs. Network Layer



- **Transport vs. Network Layer**
  - Transport layer: provides **logical communication between processes** on different hosts
  - Network layer: provides **logical communication between hosts**
  - Subtle distinction: process-to-process vs. host-to-host
- **Household Analogy**
  - Two houses (East & West Coast), each with many kids (representing processes)
  - Postal service: moves letters **house-to-house** → represents the **network layer**
  - Ann (West Coast) and Bill (East Coast): collect and distribute mail within their homes → represent the **transport layer**
  - From the cousins' (processes') perspective, Ann and Bill are the mail service



Transport Layer: End-System Services



- **Transport Layer Scope**
  - Works only at the **end systems** (like Ann and Bill inside their houses)
  - Does not participate in the **network core** (routers or postal centers)
  - Routers act only on network-layer information, ignoring transport-layer details

- **Different Service Models**
  - Substitute cousins (Susan & Harvey) collect/deliver mail less reliably → analogy for different transport protocols (e.g., TCP vs. UDP)
  - Each transport protocol offers **different reliability and service guarantees**



Discussion Question:

A client switches from UDP to TCP and attempts to connect, but the server still uses only UDP. IP datagrams can still travel between the two hosts. Can they exchange application messages? Explain why or why not.

No. The client must establish a TCP connection before exchanging application messages. The server's UDP socket cannot accept a TCP connection. IP connectivity allows packets to reach the host, but both applications must use compatible transport protocols.

Transport Layer: Capabilities and Limitations



- **Constraints from Network Layer**
  - Transport layer services are limited by what the network layer provides
  - If the network cannot guarantee delay/bandwidth, transport cannot guarantee them either

- **Additional Capabilities of Transport Layer**
  - Can still provide **reliable data transfer** even if the network is unreliable (handles loss, corruption, duplication)
  - Can offer security services (e.g., encryption for confidentiality) even if the network layer does not

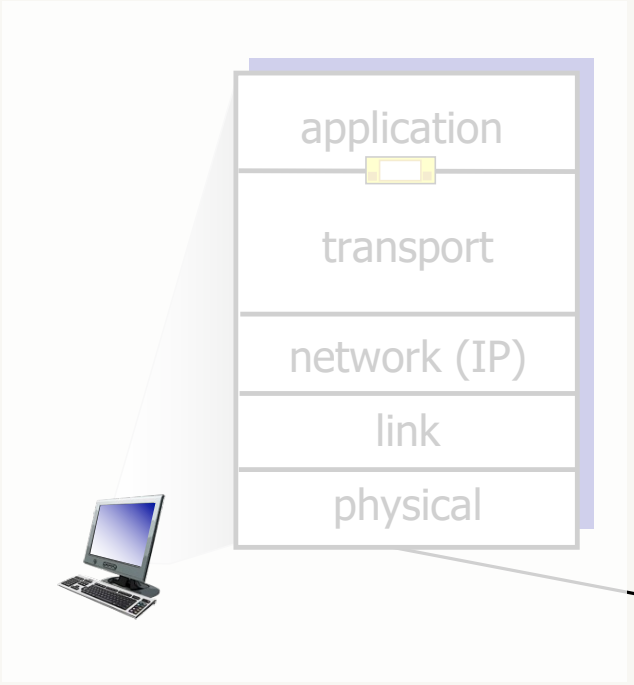
TCP and UDP: Services and Terminology



- **Transport-layer protocols in the Internet**
  - Two protocols: **UDP** (User Datagram Protocol) and **TCP** (Transmission Control Protocol)
  - UDP: **unreliable, connectionless** service
  - TCP: **reliable, connection-oriented** service
  - Developers must choose between UDP or TCP when creating sockets
- **Terminology**
  - Transport-layer packet: generally called a **segment**
  - In literature: TCP → **segment**, UDP → **datagram**
  - **To avoid confusion**: use **segment** for both TCP and UDP, and reserve **datagram** for network-layer packets

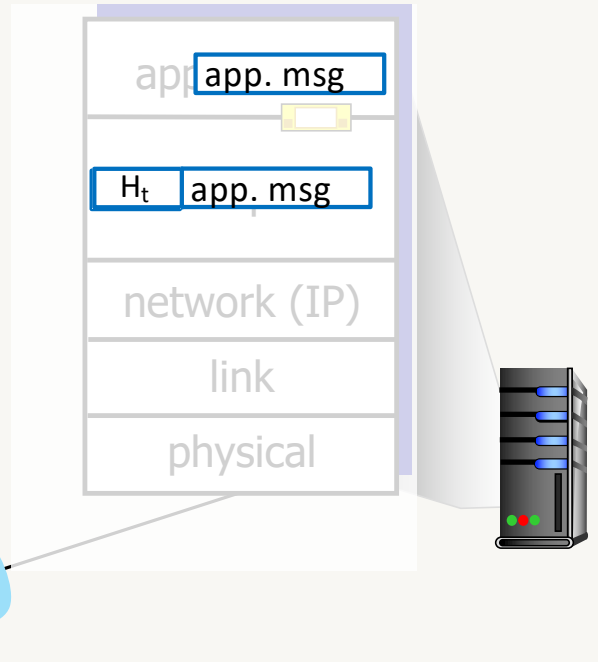
Transport-Layer Encapsulation





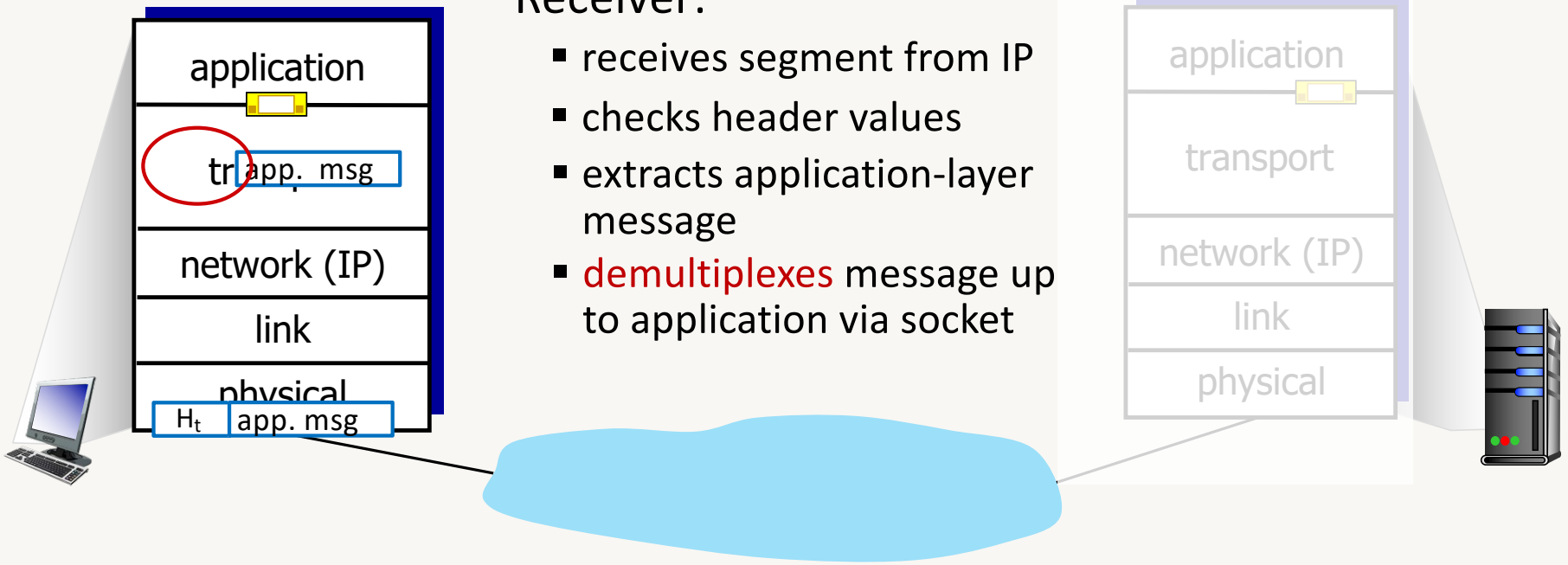
Sender:

- an application-layer message is passed
- determines segment header fields values
- creates segment
- passes segment to IP



Transport-Layer Demultiplexing





Receiver:

- receives segment from IP
- checks header values
- extracts application-layer message
- demultiplexes message up to application via socket

How UDP and TCP Extend IP Services

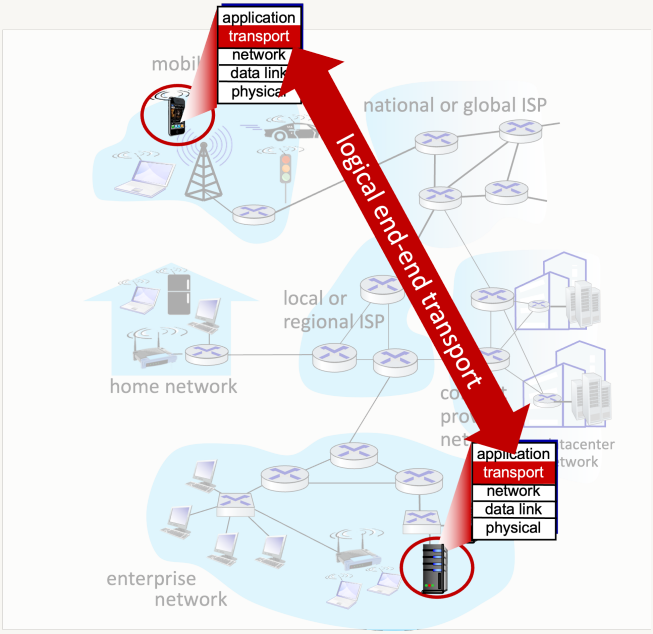


- **Internet Protocol (IP)**
  - Provides **logical communication between hosts**
  - **Best-effort delivery service**: no guarantees of delivery, order, or data integrity
  - Considered an **unreliable service**
  - Every host has at least one **IP address**
- **Fundamental role of UDP and TCP**
  - Extend IP's host-to-host delivery into **process-to-process delivery** → called **multiplexing and demultiplexing**
  - Both provide **error checking** via error-detection fields in headers

UDP and TCP: Services and Traffic Control



- **UDP services**
  - Only provides **process-to-process delivery** and **error checking**
  - No guarantee of delivery, order, or integrity
  - Essentially **unreliable** (like IP)
- **TCP services**
  - Provides **reliable data transfer** using sequence numbers, acknowledgments, flow control, and timers
  - Ensures correct, in-order delivery from sender to receiver
  - Adds **congestion control** to protect the network as a whole
  - More complex than UDP due to these mechanisms
- **Traffic characteristics**
  - **TCP traffic**: regulated (rate controlled by congestion control)
  - **UDP traffic**: unregulated (application can send at any rate)



Multiplexing and Demultiplexing



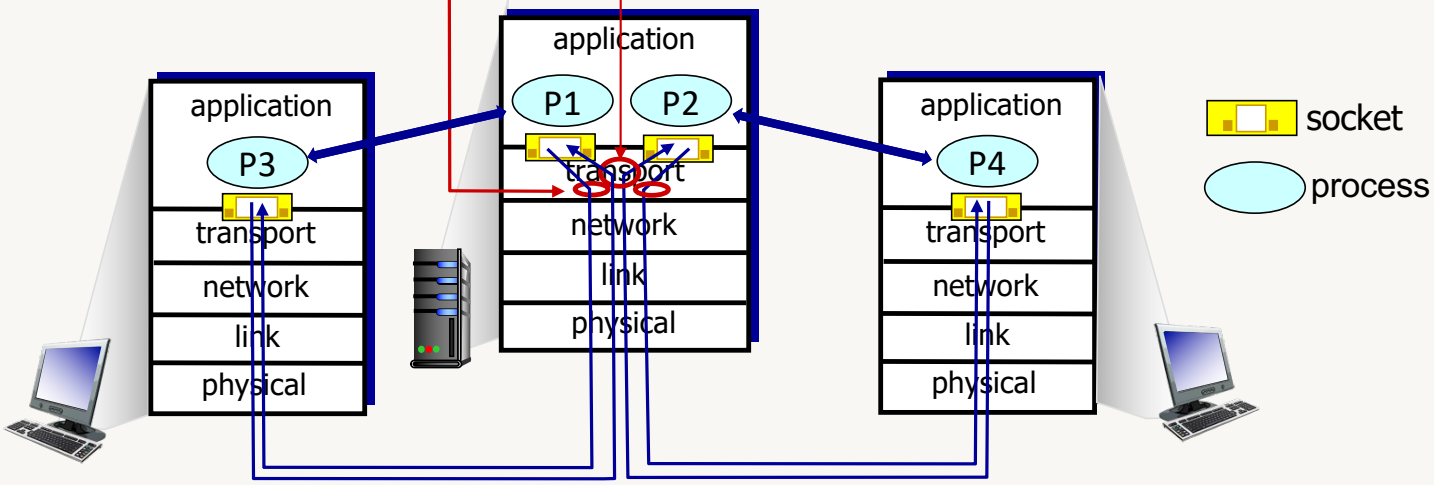
- **Purpose of multiplexing/demultiplexing**
  - Extends **host-to-host delivery (network layer)** to **process-to-process delivery (transport layer)**
  - Needed in **all computer networks**, not just the Internet

Multiplexing as sender:

Combine data from sockets, add header for delivery(later used for demultiplexing)

Demultiplexing as receiver:

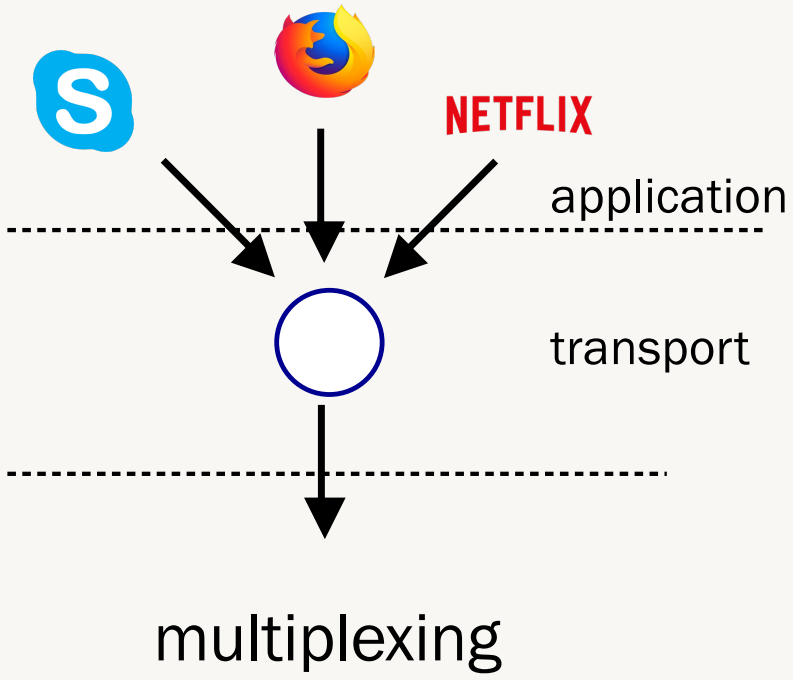
Direct segments to the right socket using header info



Multiplexing: Sending Data from Multiple Applications



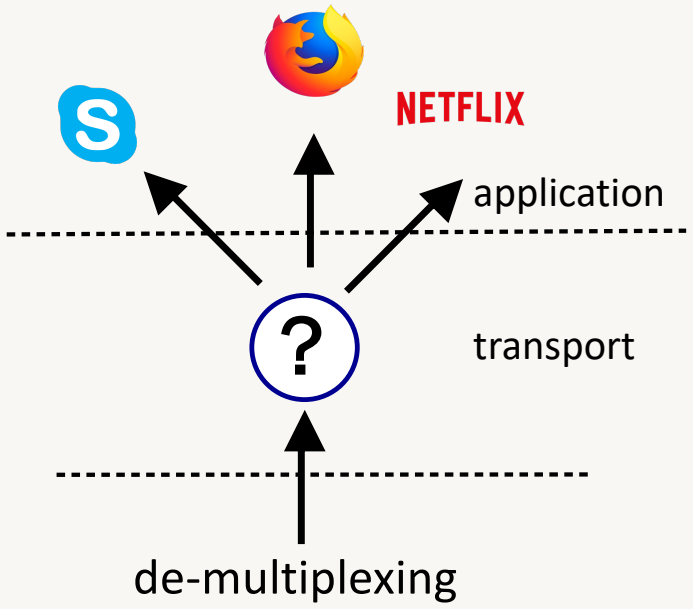
- **Multiplexing** (at sending host)
  - Transport layer gathers data chunks from multiple sockets
  - Encapsulates each chunk with header info (including port numbers)
  - Creates **segments** and passes them to the network layer



Demultiplexing: Delivering Data to the Correct Application



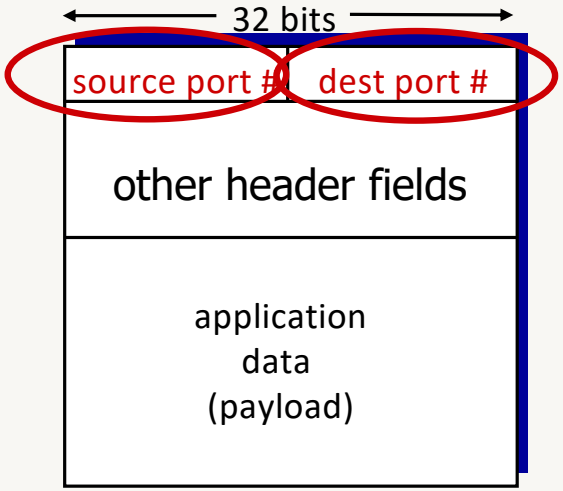
- **Demultiplexing** (at receiving host)
  - Transport layer receives segments from the network layer
  - It must deliver each segment's data to the **correct application process**
  - This is done via **sockets** (intermediary endpoints for processes)
  - Each socket has a **unique identifier** (format depends on TCP vs UDP)
  - Transport layer examines **fields in the segment** to identify the correct socket



Multiplexing and Demultiplexing Using Port Numbers



- **Household analogy**
  - Bill delivers letters to siblings based on names → **demultiplexing**
  - Ann collects letters from siblings and hands them to mail carrier → **multiplexing**
- **How it works in practice**
  - Each socket is assigned a **port number**
  - Segment header includes **source port number** and **destination port number**
  - Upon arrival, transport layer checks the **destination port number** to direct data to the correct socket and process



TCP/UDP segment format

Port Numbers and Well-Known Services



- **Port numbers**
  - Range: 0 – 65,535
  - **Well-known ports (0–1023)** reserved for common protocols (e.g., HTTP → 80, FTP → 21)
  - See Port Number Registry
  - New applications must be assigned a valid port number

Creating and Binding UDP Sockets



- Create an IPv4 UDP socket
  ```
  clientSocket = socket(AF_INET, SOCK_DGRAM)
  ```
- Without explicit binding, the system assigns an available temporary port when the socket first sends data; the port range depends on the operating system
- Use bind() to specify a local port
  ```
  clientSocket.bind(('', 19157))
  ```
- Servers bind to a port their clients know, such as UDP port 53 for DNS
- Clients usually let the system assign their local port automatically

UDP: From Sending Socket to Receiving Socket



- **UDP multiplexing/demultiplexing process**
  - Sending host includes: **source port, destination port, application data**, and other header values in the UDP segment
  - Segment is passed to the **network layer**, encapsulated in an IP datagram, and delivered best-effort
  - Receiving host checks the **destination port number** in the segment and delivers it to the correct socket

UDP Socket Identification and Reply Addressing



- **Socket identification**
  - A UDP socket is uniquely identified by a **two-tuple: (destination IP address, destination port number)**
  - Segments with different source IPs/ports but the same destination IP and port are directed to the **same socket**
- **Role of the source port number**
  - Acts as part of a **return address** for replies
  - The receiving host uses it to set the **destination port** in its response segment
  - Example: A UDP server extracts the client's source port and uses it as the destination port in its reply

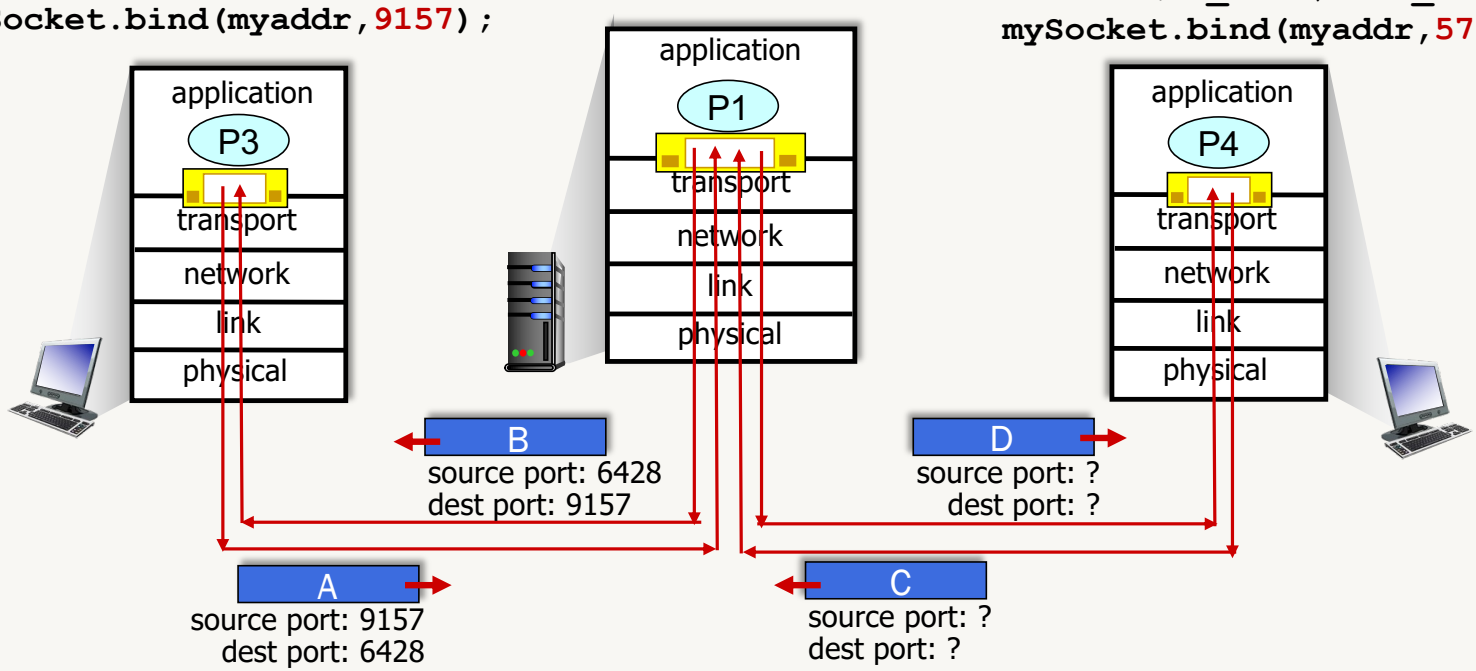
UDP: Multiple Clients, One Server Socket



```
mySocket =
 socket(AF_INET,SOCK_DGRAM)
mySocket.bind(myaddr,6428);
```

```
mySocket =
 socket(AF_INET,SOCK_DGRAM)
mySocket.bind(myaddr,9157);
```

```
mySocket =
 socket(AF_INET,SOCK_DGRAM)
mySocket.bind(myaddr,5775);
```



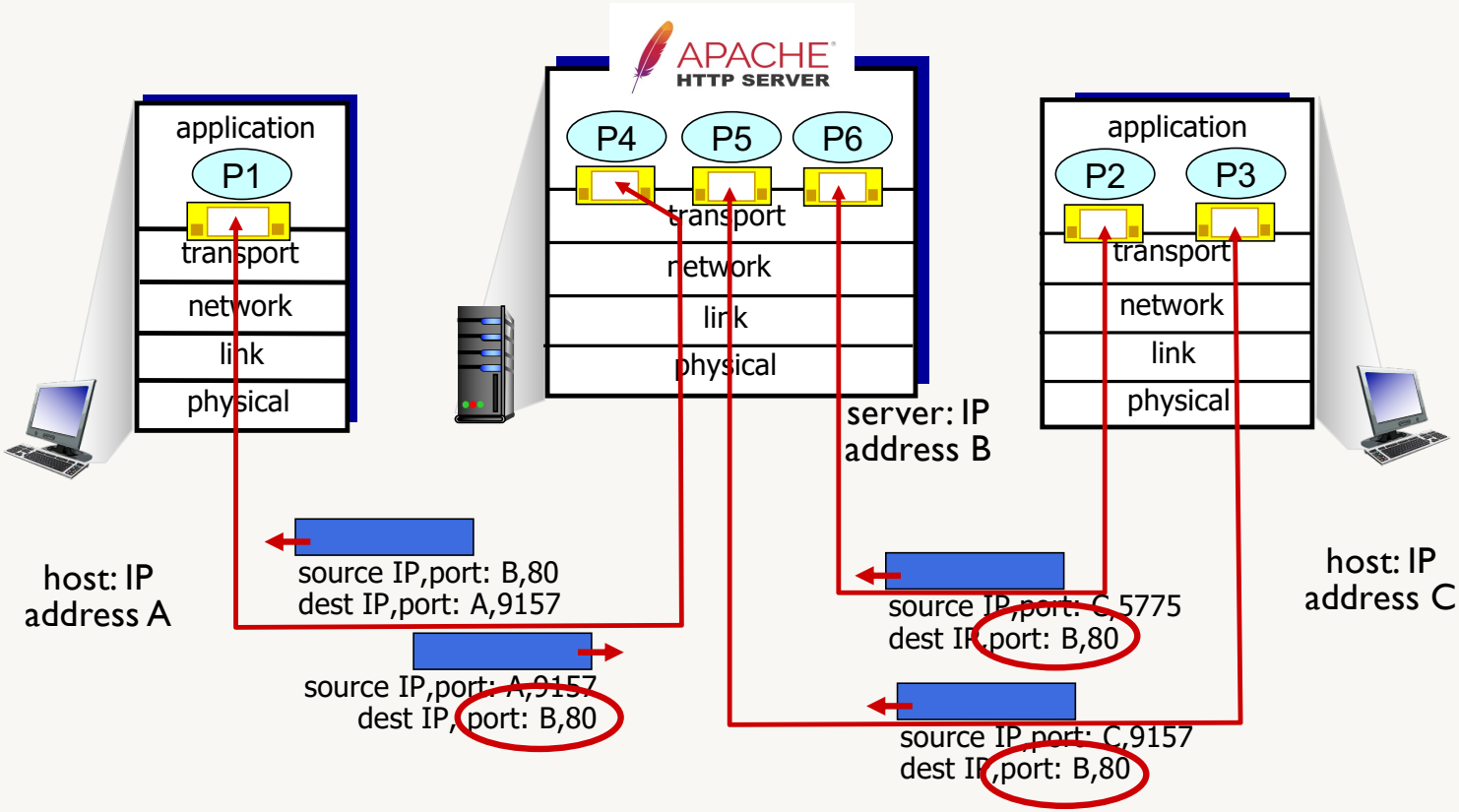
TCP Demultiplexing: Identifying Each Connection



- **TCP socket identification**
  - A TCP socket is uniquely identified by a **four-tuple**: **(source IP, source port, destination IP, destination port)**
  - This is stricter than UDP, which only uses (destination IP, destination port)
- **TCP demultiplexing**
  - A server may support many simultaneous TCP sockets
  - When a TCP segment arrives, the host checks **all four values** to direct it to the correct socket
  - Two TCP segments with the same destination IP and port but different source IPs or ports are delivered to **different sockets**

TCP: Multiple Connections, One Server Port





Three segments to IP B, port 80 are directed to different sockets



Discussion Question:

A TCP server has three established connections: (A, 9157, B, 80), (C, 5775, B, 80), and (C, 9157, B, 80). Each tuple lists the source IP, source port, destination IP, and destination port.

A segment arrives with (A, 5775, B, 80). Does it match any of these connections? Is it enough that its values appear in different connections?

No. All four values must match one connection. The incoming segment's source IP **A** matches the first connection, but its source port **5775** matches the second. TCP cannot combine values from different connections. None of the three connections matches (A, 5775, B, 80).



Discussion Question:

A client process connects to two server processes on different hosts using TCP. How many connections and client sockets does it need?

It needs two TCP connections and two connected sockets, one for each server. Each connection has its own TCP handshake. The same client process can manage both connections.



Discussion Question:

Does the client need a listening socket to communicate with these two servers?

No. The client initiates the connections using its TCP sockets. Once connected, each socket can both send and receive data. A listening socket is used to accept incoming connections in the server side.



Discussion Question:

A server has one listening socket and accepts two TCP connections. Both connections remain open. How many sockets does the server have?

Three: one listening socket and two connected sockets. Each call to accept() returns a separate connected socket. The listening socket remains available to accept more connections. (Review the Python code of the TCP client and server sockets)



Discussion Question:

Must the server create a new process for every TCP connection?

No. One server process can manage several connected sockets. The server can also use separate worker processes to handle different clients.



Discussion Question:

Can two processes run on the same server machine at the same time?

Yes. A server machine can run many processes at the same time. For example, several web server worker processes can handle different clients on the same machine.



Discussion Question:

Two clients connect to a TCP server on port 80. Must the server use a different port for each connection?

No. Both connections can use server port 80. TCP distinguishes them using all four fields: source IP, source port, destination IP, and destination port. The complete combination must be different for each connection.

For example, Server B can use port 80 for these three connections:

(A, 9157, B, 80), (C, 5775, B, 80), and (C, 9157, B, 80)

The connections from A and C can share source port 9157 because their source IP addresses differ. The two connections from C are distinguished by their source ports, 5775 and 9157.

References



- Ch. 3, Kurose and Ross



Questions?